Title

SQL Queries Demonstrating All Types of Joins, Subqueries, and Views

---

Learning Objectives

- To create and use a sample database for applying SQL queries.
- To implement **all types of joins**: INNER, LEFT, RIGHT, FULL, SELF.
- To practice **subqueries** (single-row, multiple-row, EXISTS).
- To create and use **views** for simplified query results.

Introduction

SQL provides powerful features to extract and combine data from multiple tables:

- **Joins** (INNER, OUTER, SELF) combine related rows from different tables.
- **Subqueries** allow nesting one query inside another for advanced filtering.
- **Views** provide virtual tables for simplified data access.

In this experiment, we create two tables: **Employees** and **Departments**, insert sample data, and run queries to demonstrate joins, subqueries, and views.

1. **Inner Join**

Returns only rows with matching values in both tables.

Syntax:

SELECT columns

FROM table1

INNER JOIN table2

ON table1.col = table2.col;

Example:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employees e

INNER JOIN Departments d

ON e.dept_id = d.dept_id;

---

2. Left Join

Returns all rows from the left table and matching rows from the right table.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.col = table2.col;
```

Example:

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_id = d.dept_id;
```

---

3. Right Join

Returns all rows from the right table and matching rows from the left table.

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.col = table2.col;
```

Example:

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_id = d.dept_id;
```

---

4. Full Outer Join

Returns all rows when there is a match in either left or right table (MySQL uses UNION of LEFT and RIGHT join).

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.col = table2.col
UNION
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.col = table2.col;
```

Example:

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_id = d.dept_id;
```

---

5. Self Join

Joins a table with itself to represent hierarchical data.

Syntax:

```
SELECT a.col, b.col
FROM table a
```

```
LEFT JOIN table b
ON a.related_id = b.id;
```

Example:

```
SELECT e.emp_id AS Employee_ID,
      e.emp_name AS Employee_Name,
      m.emp_name AS Manager_Name
FROM Employees e
LEFT JOIN Employees m
ON e.manager_id = m.emp_id;
```

---

6. Subquery (=)

A subquery returns a single row; used with =, <, > operators

Syntax:

```
SELECT columns
FROM table
WHERE col = (SELECT col FROM table WHERE condition);
```

Example:

```
SELECT emp_name
FROM Employees
WHERE dept_id = (
   SELECT dept_id FROM Departments WHERE dept_name = 'HR role'
);
```

---

7. Subquery with IN

Used when subquery returns multiple rows; used with IN, ANY, ALL operators

Syntax:

```
SELECT columns
FROM table
```

WHERE col IN (SELECT col FROM table WHERE condition);

Example:

```
SELECT emp_name
FROM Employees
WHERE dept_id IN (
   SELECT dept_id FROM Departments WHERE dept_name IN ('HR role','Sales role')
);
```

---

8. Subquery with EXISTS

Checks if the subquery returns any rows.

Syntax:

```
SELECT columns
FROM table1
WHERE EXISTS (SELECT 1 FROM table2 WHERE condition);
```

Example:

```
SELECT emp_name
FROM Employees e
WHERE EXISTS (
   SELECT 1 FROM Departments d WHERE d.dept_id = e.dept_id
);
```

---

9. View Creation

A view is a virtual table created from a query.

Syntax:

```
CREATE VIEW view_name AS
SELECT columns
FROM table1
JOIN table2 ON condition;
```

Example:

```
CREATE VIEW EmployeeDetails AS

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employees e

JOIN Departments d ON e.dept_id = d.dept_id;
```

---

10. Select from View

Retrieves data from the created view like a normal table.

Syntax:

```
SELECT * FROM view_name;
```

Example:

```
SELECT * FROM EmployeeDetails;
```

Conclusion

- We created and executed queries demonstrating **all joins (INNER, LEFT, RIGHT, FULL, SELF)**.
- We implemented **subqueries with =, IN, and EXISTS**.
- We created and queried a **view** for simplified data access.
- This practical strengthened our understanding of advanced SQL query techniques.